UiT The Arctic University of Norway

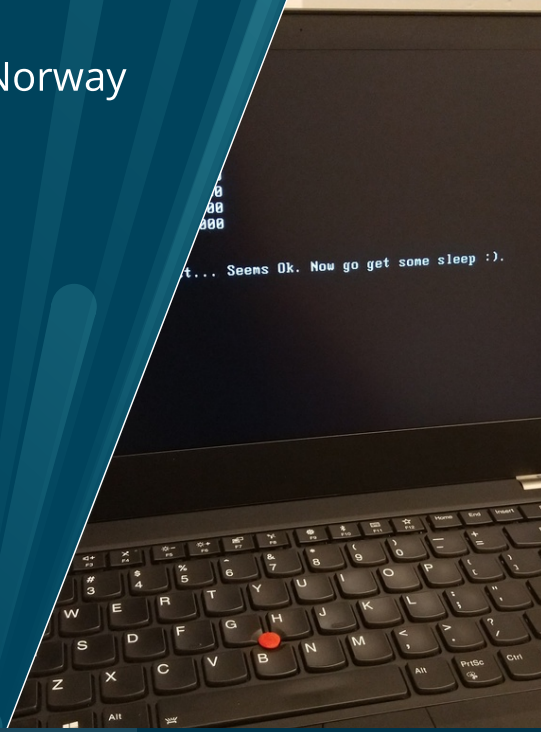
Project 2

Protection and Syscalls

Mike Murphy

UiT INF-2203

Spring 2026



P2: Protection and Syscalls

- P1: You can boot and run a program
- P2: What happens if a program misbehaves?

1. Configure CPU to catch *exceptions*
   - Something fishy happens, e.g. div by zero, bad opcode
   - CPU will call a kernel function
2. Turn on CPU *memory protection*
   - Forbid hardware access
   - Forbid kernel mem
   - CPU will not allow access
   - Cause exception instead
3. Force processes to make requests via *system calls*
   - If a process can't access hardware, how can it do I/O?
   - Kernel provides *system calls*

New Test Programs

divzero-raw

- Divides by zero
- Causes an exception
- Need to handle exceptions

eraser-raw

- Overwrites kernel/program memory
- Need to protect kernel mem

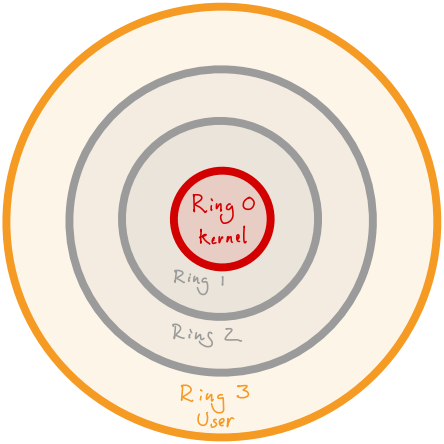
```
# Try to erase self
eraser-raw 0x500000:0x1000
# Try to erase kernel
eraser-raw 0x10000:0x8000
```

Portable (not-raw) Programs

- `noop`, `hello`, `plane` (`divzero`, `eraser`)
- Classic C:
  - `main()`
  - `printf()`
  - etc.
- Link to standard library
  - Provides init: `_start` → `main`
  - Provides I/O via *syscalls*

x86 Protection Rings

- x86 has four protection levels
- Inner rings are most privileged
  - **Ring 0: kernel level**
  - Ring 1: ?
  - Ring 2: ?
  - **Ring 3: *user* level**
- Rings 1&2?
  - Intended for drivers
  - Rarely used in practice
- Rings configured via *Global Descriptor Table (GDT)*



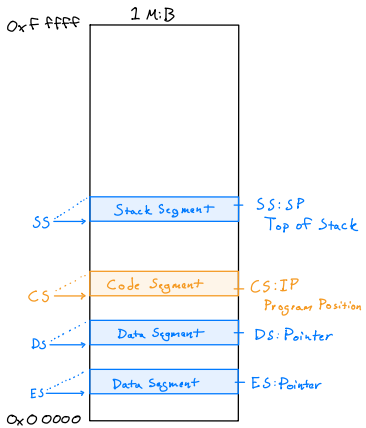
Privilege levels

GDT History: 8086 Segment Registers

- Intel 8086 (1978)
  - 16-bit registers (64 KiB)
  - 20-bit address space (1 MiB)
- *Segment* registers: *Base* for access

```
Code Segment (CS)              0x 8000
Instruction Pointer (IP)      +0x  1234
                              ---------
Actual instruction             0x 81234
```

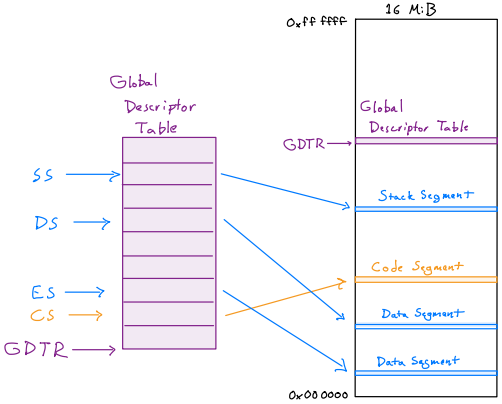
Segment Reg	Base for
CS: Code Segment	IP: Instruction Pointer
SS: Stack Segment	SP: Stack Pointer
DS: Data Segment	data pointers
ES: Extra Segment	data pointers



8086 Addressing

GDT History: 286 Upgrade

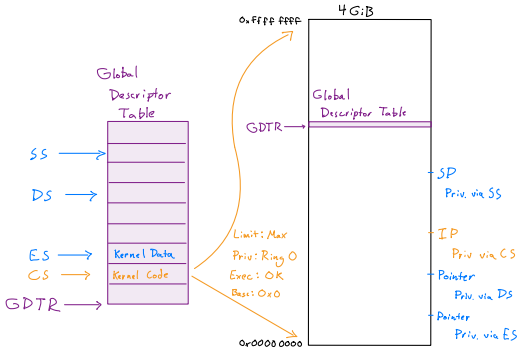
- Intel 286 (1982)
  - 16-bit registers (64 KiB)
  - 24-bit address space (16 MiB)
- Global Descriptor Table
  - Move segment info to RAM
  - Table of segment *descriptors*
  - New register GDTR: points to table
  - Segment regs become *index* into table
- Descriptor data
  - Base address
  - Size
  - **Privilege level**
  - Additional flags



286 Addressing

GDT History: 386 Flat Addressing

- Intel 386 (1985)
  - 32-bit registers (4 GiB)
  - 32-bit address space (4 GiB)
- *Flat addressing:* direct 32-bit pointers
  - GDT becomes semi-vestigial
  - Base & size: 0 → max
  - But **Type and privilege still matter**
- Typical: 4 segments
  - (code vs data) × (kernel vs user)
  - CS → code seg w/ ring 0
    Can execute privileged instructions
  - DS → data seg w/ ring 3
    Can only access user memory



386 Addressing

Interrupts and Exceptions

- *Interrupt:* Event needs response
  - CPU *interrupts* normal execution
  - Runs a designated *Interrupt Service Routine (ISR)*
  - Returns to normal execution
- Types of interrupts
  - *Interrupt Request (IRQ):* Hardware wants attention
  - *Exception:* CPU operation fails (e.g. divide by zero)
  - *Trap:* CPU operation is flagged (e.g. breakpoint)
- Trap vs exception?
  - Interrupt: stop *between* ops, resume at next op
  - Exception: try op, *op fails*, resume *after* failed op
  - Trap: stop *before* op, make adjustments, then try op
- x86: 256 *Interrupt Vectors*
  - 32 reserved by CPU

Important x86 Exception Vectors

Vec	Abbr	Desc	Example
0	#DE	Divide Error	Div by zero
6	#UD	Undefined Opcode	Jumped to data/garbage
8	#DF	Double Fault	Other error with no ISR
13	#GP	General Protection Fault	User code restriction

x86 Interrupt Descriptor Table

- 8086: Interrupt *Vector* Table
  - Simple array in RAM starting at address 0x0
  - 256 entries: each with 16-bi CS and IP
  - OS sets up table to point to ISRs
    - Vector 0 → CS:IP at 0x0 → Div zero ISR
    - Vector 1 → CS:IP at 0x4 → Debug trap ISR
    - …
- 286: Interrupt *Descriptor* Table
  - Expands from vector to *descriptor*
  - New register IDTR: points to table
    - Vector 0 → descriptor at IDTR + 0x0 → Div zero ISR
    - Vector 1 → descriptor at IDTR + 0x8 → Debug trap ISR
    - …
  - CS field now determines privilege (GDT index)
  - IP field expands to 32-bits
  - Additional fields

If Table is Not Set Up

1. Interrupt or exception
   - First level
   - If ISR is set up properly, good
   - If not…
2. Double Fault (#DF, exception #8)
   - Fallback exception
   - Called if first exception handler can't be executed
   - If #DF ISR up correctly, good
   - If not…
3. Triple Fault
   - Fallback failed
   - Nothing left to do
   - Reboot or halt
   - May be able to configure reboot vs halt at emulator level (QEMU)

Regular Function vs ISR

Function

- Call with CALL
- Stack:
  - Caller pushes params
  - CPU pushes IP return addr
- Return with RET

```
int example_fn(
        int x,
        int y)
{ return x+y; }
```

x86 Interrupt Service Routines (ISRs)

- Trigger condition, or INT op
- Stack:
  - No params
  - CPU pushes SS, SP,[1] FLAGS, CS, IP,
  - May also push error code[2]
- Return with IRET

```
INTERRUPT_HANDLER
void example_isr(
        struct x86_intr_frame *frame,
        ureg_t errcode)
{ return; }
```

[1] Stack regs are only pushed if switching to higher priv

[2] ISR function signature must reflect error code vs. no error code

x86 Interrupt Detail

1. Hardware, exception, or INT op triggers interrupt
2. CPU determines vector, looks in IDT
3. CPU checks privilege
   - IDT entry includes a CS value
   - CS value is index into GDT
   - GDT entry's privilege field determines new privilege
4. CPU pushes current state
   - SS, SP (only if privilege change)
   - FLAGS, CS, IP (core program state)
   - Error code (only if exception has error code)
5. CPU jumps to ISR
   - If privilege change, updates CS and SS (switch to kernel priv level)
   - If privilege change, updates SP (switch to kernel stack)
   - Jumps to requested IP

New Stack?

- Where to jump to:
  - CS/IP comes from IDT entry
  - New CS determines new privilege level
- What stack to use?
  - Want separate stacks for different priv levels
  - Security: don't write kernel data to user-accessible stack
  - How does CPU pick stack to switch to?
- **Task State Segment**
  - Special segment in GDT: GDT entry points to structure in memory
  - Semi vestigial, like GDT
    - Has fields for every register
    - Intent was for full CPU support for task switching
    - But most OSes do their own task switching
  - Only use stack fields: SS and SP for switch to ring 0

Summary So Far

To set up privilege level switching and enforcement, you need…

1. **Global Descriptor Table (GDT):** Configure priv levels
   - Create GDT with four segment descriptors
   - Kernel code, kernel data, user code, user data
2. **Task State Segment (TSS):** Configure what kernel stack to use
   - Reserve memory for one TSS
   - Add fifth GDT entry to point to it
   - When switching to user level,
     save desired kernel stack SS and SP to fields in TSS
3. **Interrupt Descriptor Table (IDT):** Configure exception handling
   - Create ISR functions in kernel code
   - Create IDT to point CPU to target ISRs
   - e.g. #DE Divide Error, #GP General Protection Fault

Summary So Far

To set up privilege level switching and enforcement, you need…

1. **Global Descriptor Table (GDT):** Configure priv levels
   - Create GDT with four segment descriptors
   - Kernel code, kernel data, user code, user data
2. **Task State Segment (TSS):** Configure what kernel stack to use
   - Reserve memory for one TSS
   - Add fifth GDT entry to point to it
   - When switching to user level,
     save desired kernel stack SS and SP to fields in TSS
3. **Interrupt Descriptor Table (IDT):** Configure exception handling
   - Create ISR functions in kernel code
   - Create IDT to point CPU to target ISRs
   - e.g. #DE Divide Error, #GP General Protection Fault
4. **Page Tables:** Set memory permissions

Divide Memory into *Pages*

- 286: GDT handled permissions
  - Different segments could have different read/write permissions
  - Idea: many segments for many different permission regions
- 386: Segments cover whole address space
  - Few segment types: kernel code/data, user code/data
  - What about memory permission regions?
- **Pages:** Divide memory into uniform chunks
  - Set properties per page
  - Common size: 4 KiB
    - Matches common disk block size for easy loading
    - Notice that ELF segments start on 4 KiB boundaries
- *Page Tables* in memory determine page properties
  - One entry for every page
  - 4 GiB address space / 4 KiB page size = 1 Mi pages (1024x1024)

Page Tables are a Tree

- Hierarchy
  - A CPU register points to root of tree
  - Root table points to child tables
  - Final table points to page itself
- 386 has two levels of tables
  - Register **CR3** points to *Page Directory*
  - **Page Directory** points to up to 1024 *Page Tables*
  - Each **Page Table** points to up to 1024 *Pages*
  - Each **Page** has 4096 bytes

  CR3 $\rightarrow$ Page Directory $\rightarrow$ Page Table $\rightarrow$ Page
  - 1024 x 1024 x 4096 B = 4 GiB address space
- Larger address spaces add more levels of hierarchy

Virtual Memory

- Page Tables are the heart of *Virtual Memory (VMem)*
- Not only permissions
- Can also copy/move pages to other parts of memory
  - e.g. can move page 0x1000 to real address 0xa000 but still access it as 0x1000
- Can also *swap out* pages to disk
  - Get *Page Fault Exception* if page is swapped out
  - Can *swap in* and resume
- Don't worry about VMem for this assignment
  - We will only adjust permissions
  - Precode provides functions to help

Summary #2

- To defend against `divzero-raw`
  - Need to set up IDT
  - Need to catch exception and return to kernel shell
- To defend against `erase-raw`
  - Need to change privilege level when launching process
    - Need to set up GDT and TSS
    - Need to update process launch procedure
  - Need to change page permissions
    - Set up Page Tables
    - Mark kernel pages as kernel level
    - Mark process pages as user level
- DON'T PANIC
  - Precode handles a lot of the GDT/IDT/etc. details

Configuring Permissions Breaks Other Programs

- When you have protection enabled
  - Raw programs stop working
  - They cannot access their arguments on stack
  - They cannot *return* to calling code
  - `hello-raw` cannot access I/O ports
  - `plane-raw` cannot draw to screen
- Need to set up *C Runtime* and *System Calls*

System Calls

- Processes cannot do their own I/O
  - Need ask kernel
  - How to ask kernel?
- **System Calls**
  - CPU provides a way to escalate privilege: *interrupts*
  - (Some CPUs have special opcodes, but we will use interrupt)
  - INT instruction triggers an interrupt with software
  - We will choose an interrupt vector to use
    - Linux i386 uses INT 0x80 (dec 128)
    - We will use INT 0x30 (dec 48)

System Call Implementation

- All syscalls go through same interrupt
  - Each syscall has a *syscall number*
  - Syscall number is first parameter
- On process side:
  1. `printf` calls `write`
  2. `write` calls `syscall`
  3. `syscall` places args and invokes `INT 0x30`
- On kernel side:
  1. ISR collects args, passes to *dispatch*
  2. Dispatch determines appropriate action, calls function
  3. Syscall function performs action, returns result

Syscall ABI

Reg	Arg
EAX	Syscall Nr
EBX	Arg 1
ECX	Arg 2
EDX	Arg 3
ESI	Arg 4
EDI	Arg 5

- INT 0x30
- Return val in EAX

Process Start

- P1 process launch was a simple function call
  - Same stack. Arguments on stack.
  - Call `_start`, `_start` returns.
- P2 launch is more complicated
  - Switch from kernel priv to user priv
  - User level needs its own stack
- Kernel side:
  - Allocates space for process's stack
  - Copies arguments to process's stack
  - Transfers to process code with user privilege
- User side:
  - `_start` process arguments to `argc` and `argv`
  - `_start` calls `main`

Process Exit

- When `main` returns...
  - Stack ends
  - `_start` does not have a return address

Process Exit

- When `main` returns...
  - Stack ends
  - `_start` does not have a return address
  - Solution: call `_exit` syscall

Process Exit

- When `main` returns…
  - Stack ends
  - `_start` does not have a return address
  - Solution: call `_exit` syscall
- Typical `_start`:
  1. Translate process arguments
  2. Pass `argc` and `argv` to `main`
  3. When main returns, call `_exit` syscall

C Runtime / Standard Library

Raw C Program

```c
int _start(int argc, char* argv[]) {
    /* ... Do raw I/O ... */
    return 0;
}
```

Familiar C Program

```c
#include <stdio.h>

int main(int argc, char* argv[]) {
    printf("Hello, world!\n");
    return 0;
}
```

C Runtime / Standard Library

Raw C Program

```c
int _start(int argc, char* argv[]) {
    /* ... Do raw I/O ... */
    return 0;
}
```

Familiar C Program

```c
#include <stdio.h>

int main(int argc, char* argv[]) {
    printf("Hello, world!\n");
    return 0;
}
```

- Kernel
  1. Loads program
  2. Transfers control to `_start`
  3. Provides syscalls like `write`, `_exit`

C Runtime / Standard Library

Raw C Program

```c
int _start(int argc, char* argv[]) {
    /* ... Do raw I/O ... */
    return 0;
}
```

Familiar C Program

```c
#include <stdio.h>

int main(int argc, char* argv[]) {
    printf("Hello, world!\n");
    return 0;
}
```

- Kernel
  1. Loads program
  2. Transfers control to `_start`
  3. Provides syscalls like `write`, `_exit`
- C Runtime (`crt0.o`)
  - crt0 = C Runtime Zero
  - Provides `_start` to call `main`, handle `_exit`

C Runtime / Standard Library

Raw C Program

```c
int _start(int argc, char* argv[]) {
    /* ... Do raw I/O ... */
    return 0;
}
```

Familiar C Program

```c
#include <stdio.h>

int main(int argc, char* argv[]) {
    printf("Hello, world!\n");
    return 0;
}
```

- Kernel
  1. Loads program
  2. Transfers control to `_start`
  3. Provides syscalls like `write`, `_exit`
- C Runtime (`crt0.o`)
  - crt0 = C Runtime Zero
  - Provides `_start` to call `main`, handle `_exit`
- Standard Library (`libc.a`)
  - Provides standard APIs
  - e.g. `printf`, implemented via syscalls

C Runtime / Standard Library

Raw C Program

```c
int _start(int argc, char* argv[]) {
    /* ... Do raw I/O ... */
    return 0;
}
```

Familiar C Program

```c
#include <stdio.h>

int main(int argc, char* argv[]) {
    printf("Hello, world!\n");
    return 0;
}
```

- Kernel
  1. Loads program
  2. Transfers control to `_start`
  3. Provides syscalls like `write`, `_exit`
- C Runtime (`crt0.o`)
  - crt0 = C Runtime Zero
  - Provides `_start` to call `main`, handle `_exit`
- Standard Library (`libc.a`)
  - Provides standard APIs
  - e.g. `printf`, implemented via syscalls
- Familiar portable program (e.g. `hello.c`)
  - Uses `main` and `printf`
  - Auto-linked with `libc.a` and `crt0.o`

Project 2 Tasks Summary

We Provide

- Helper code for CPU data structures (GDT, IDT, etc.)
- Simple `crt0` and `libc` implementations (Munix libc = `mulibc`)

Your Task: **Protect Kernel, Provide Syscalls**

1. Configure CPU for protection and exception handling
2. Protect kernel from misbehaving processes (divzero and eraser)
3. Implement syscalls so that portable processes work (new hello and plane)

Suggested Task Order I

1. Turn on exception handling
   - Set up IDT
   - Catch Double Fault first (no more instant reboot)
   - Then catch Divide Error (specifically handle divzero)
2. Turn on protection
   - Play around with `eraser-raw`, see error behavior
   - Set up GDT and TSS
   - Allocate user stack for user process
   - Change process launch to switch to user priv level
   - When you get it right, `hello-raw` will cause General Protection Fault
3. Implement syscalls
   - Implement `_exit` (new `noop` should return to kernel)
   - Implement `write` via kernel's `file_read`
     (new `hello` should output to serial)

Suggested Task Order II

4. Fix argument passing
   - Copy process args to user stack before launch
   - Hint: Use *System V ABI* (see PDF in `doc/abi/`)
   - New `hello`, `plane`, and `eraser` should be able to read their args
5. Turn on page protection
   - Enable Page Table structure
   - Mark kernel pages as kernel-level (“supervisor”)
   - Mark process pages as user-level (“user”)
   - Extra: compare to ELF segments, set read-only segment as read-only
   - Finally, `eraser-raw` and `eraser` won’t be able to crash kernel, only themselves

New in Precode

- More architecture-specific code (in `lib/arch`)
  - Helper code for managing CPU structures
    - Macros for creating ISRs
  - Helper code for decoding descriptors and error codes
- Simple `libc` and `crt0.o` implementations
  - Portable code (C only) in `lib/mulibc`
  - Arch-specific `crt0` and syscall code in `lib/sys/i386-munix/`
  - Also implemented: `lib/sys/x86_64-linux/`
    - Test programs can compile for Linux!
    - Run `make test`
    - Run `out/x86_64-linux/hello`
    - So far, only compiles for Linux and Munix (Sorry, Mac!)

Two Ways to Continue

1. **Continue** building on your solution
   - Git branch `main`
   - Starts with P1 precode, adds only new P2 precode (no P1 solution)
   - Should *(should!)* merge cleanly with your P1: `git merge main`
   - More challenging, but more rewarding. OS is more *yours*
   - More chances for bugs. Will have to do more troubleshooting
2. **Start fresh** with common P2 base
   - Git branch `p2-pre`
   - Starts with my P1 solution, then adds new P2 precode
   - Check out, do not merge: `git checkout p2-pre`
   - Use common project base, focus only on current project
   - A more straightforward project experience

- Remember: This is new code. **There will be bugs either way!**

P2: New in precode

new	pre	task	file	desc
!	13		process/divzero-raw.c	Exception process
!	188		process/eraser-raw.c	Malicious process
!	13		process/divzero.c	Portable divzero
!	218		process/eraser.c	Portable eraser
!	1		process/noop.c	Portable no-op
!	12		process/hello.c	Portable hello
!	147		process/plane.c	Portable plane
!	131		lib/mulibc/	Tiny standard lib
!	21		lib/sys/	crt0/syscall impls
!	475	4	lib/arch/i386/x86_seg.[ch]	GDT/TSS
!	380	29	lib/arch/i386/cpu_interrupt.[ch]	Interrupts/IDT
!	19		lib/arch/i386/syscall_entry.S	System call ISR
!	238		lib/arch/i386/cpu_pagemap.[ch]	Page Tables
!	31		kernel/interrupt.c	Interrupt dispatch
!	35	2	kernel/syscall_dispatch.c	Syscall dispatch
!	305		kernel/pagemap.[ch]	Page Table Helpers

P2: Updated in precode

new	pre	task	file	desc
	37		Makefile.template	Build portable processes
	9		lib/arch/i386/abi.h	Mem constants, PUSH macro
	24		lib/arch/i386/cpu.h	Priv level, new fns
	18	1	kernel/kernel.[ch]	kernel_noreturn
	5		kernel/kshell.[ch]	connect process files to shell
	100	45	kernel/process.[ch]	skeleton of process task

P2: Task files

new	pre	task	file	desc
!	380	29	lib/arch/i386/cpu_interrupt.[ch]	Enable ISRs
!	475	4	lib/arch/i386/x86_seg.[ch]	Create GDT segments
	100	45	kernel/process.[ch]	Enhance processes
!	35	2	kernel/syscall_dispatch.c	Route syscalls
	18	1	kernel/kernel.[ch]	Turn on page mapping

Estimated project workload: 81 lines

NB: Don't just treat this project as a fill-in-the-blanks exercise.
It is important that you understand how the pieces fit together.

Hints

- Debugger (`gdb`)
  - Also add process file to debug info

```
(gdb) add-symbol-file process/eraser-raw
add symbol table from file "process/eraser-raw"
(y or n) y
```

    Note that debug runs in `out/i386-elf`

- Keep using `readelf` and `objdump`
  - Get used to comparing C file to compiled assembly (`objdump -d out/i386-elf/process/hello`)
- Exception in log? Look at exception IP value
  - Use `objdump -d` and find offending instruction in `kernel` or process file

Reference Manuals

- CPU manuals[3]
  - Look for sections that focus on 32-bit
    - AMD usually calls it “Legacy Mode” vs 64-bit “Long Mode”
    - Intell usually calls it “Protected Mode” vs 64-bit “IA-32e Mode”
- System V ABI (PDF in `doc/abi/`)
  - Munix and Linux ABIs are based on this
  - Gives details on passing parameters, setting up process stack, etc.
- C Standard Library reference: https://en.cppreference.com/w/c.html
- Linux syscall man pages
  - `man syscall` (singlar): info on syscall ABI
  - `man syscalls` (plural): list of provided syscalls
  - `man 2 write`: write call behavior
  - `man 2 _exit`: exit call behavior

[3] Intel manuals: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
AMD manuals: https://docs.amd.com/

Suggested Extra Challenges

- Write more programs
- Implement standard input: `fread` via `read` syscall
  - Advanced: Create a keyboard driver using keyboard interrupt
- Mac user? Help us implement syscalls on Mac
  so that you can compile and run the test programs too

Hand-In Format: Same as Last Time

- Design reviews mid-way
  - Meet with TA
  - Convince them you have started and have a plan for solving this
- Hand in: Code and Report
  - Zip up code tree + git repo, upload to Canvas
    - Be sure to clean your code tree: `make clean`
    - DO NOT include cross compiler or bootloader
    - DO NOT include `out/` directory
    - DO include `.git/` directory
    - Zip should be a handful of Megabytes
    - Cross compiler will balloon it to Gigabytes (no thanks!)
  - Report
    - See Report Guidelines doc for detail
    - Upload report PDF to Canvas separately from code Zip